

10. Hafta - Karar Ağaçları (Decision Tree) Modeli

May 12, 2025

1 10. Hafta: Karar Ağaçları (Decision Tree) Modeli

7.1. Decision Tree modelinin uygulanması ve hiperparametrelerinin açıklanması

7.2. Modelin eğitilmesi - Özellik ölçekleme yöntemlerinin uygulanması

7.3. Model sonuçlarının değerlendirilmesi

1.0.1 7.1. Decision Tree modelinin uygulanması ve hiperparametrelerinin açıklanması

```
[1]: # Python'da dengesiz veri setleriyle çalışmak için kullanılan imbalanced-learn  
      ↳ adlı bir kütüphaneyi yükleyelim.  
  
      # pip install imbalanced-learn
```

imbalanced-learn Nedir?

Makine öğrenmesinde, bazı sınıfların diğerlerine göre çok daha fazla örneğe sahip olduğu dengesiz veri setleri (imbalanced

datasets) sıkça karşımıza çıkar. Bu durum, modelin baskın olan sınıfa daha fazla ağırlık vermesine ve azınlık sınıfı görmezden

gelmesine neden olabilir.

imbalanced-learn, bu tür veri setlerini dengelemek için çeşitli teknikler sunar. Özellikle sınıflar arasındaki örnek sayısını

eşitlemek için over-sampling (azınlık sınıfını artırma) ve under-sampling (çoğunluk sınıfını azaltma) yöntemlerini içerir.

Temel Özellikleri

- SMOTE (Synthetic Minority Over-sampling Technique)
- RandomOverSampler ve RandomUnderSampler
- TomekLinks, NearMiss, EditedNearestNeighbours gibi gelişmiş teknikler
- Pipeline ile scikit-learn entegrasyonu

Bu algoritma özelinde RandomUnderSampler yöntemi kullanılacaktır.

```
[2]: # Gerekli kütüphanelerin import edilmesi
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, \
    cross_validate, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    f1_score, confusion_matrix, classification_report
from imblearn.under_sampling import RandomUnderSampler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
[3]: # Veri setini oku
df = pd.read_csv("cleaned_df_data.csv")
```

```
[4]: # burada kaç veri var
df.shape
```

```
[4]: (45211, 15)
```

```
[5]: df
```

```
[5]:
```

	age	job	marital	education	default	housing	loan	\
0	58	management	married	tertiary	0	1	0	
1	44	technician	single	secondary	0	1	0	
2	33	entrepreneur	married	secondary	0	1	1	
3	47	blue-collar	married	unknown	0	1	0	
4	33	unknown	single	unknown	0	0	0	
...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	0	0	0	
45207	71	retired	divorced	primary	0	0	0	
45208	72	retired	married	secondary	0	0	0	
45209	57	blue-collar	married	secondary	0	0	0	
45210	37	entrepreneur	married	secondary	0	0	0	

	contact	month	campaign	previous	poutcome	y	balance_level	\
0	unknown	may	1	0	unknown	0	2	
1	unknown	may	1	0	unknown	0	1	
2	unknown	may	1	0	unknown	0	1	
3	unknown	may	1	0	unknown	0	1	
4	unknown	may	1	0	unknown	0	1	
...	...	...	...	...	...	...	...	
45206	cellular	nov	3	0	unknown	1	1	
45207	cellular	nov	2	0	unknown	1	1	
45208	cellular	nov	5	3	success	1	2	

45209	telephone	nov	4	0	unknown	0	1
45210	cellular	nov	2	11	other	0	2

has_any_credit	
0	1
1	1
2	1
3	1
4	0
...	...
45206	0
45207	0
45208	0
45209	0
45210	0

[45211 rows x 15 columns]

```
[6]: df.isna().any().sum()
```

```
[6]: 0
```

```
[7]: df
```

```
[7]:
```

	age	job	marital	education	default	housing	loan	\
0	58	management	married	tertiary	0	1	0	
1	44	technician	single	secondary	0	1	0	
2	33	entrepreneur	married	secondary	0	1	1	
3	47	blue-collar	married	unknown	0	1	0	
4	33	unknown	single	unknown	0	0	0	
...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	0	0	0	
45207	71	retired	divorced	primary	0	0	0	
45208	72	retired	married	secondary	0	0	0	
45209	57	blue-collar	married	secondary	0	0	0	
45210	37	entrepreneur	married	secondary	0	0	0	

	contact	month	campaign	previous	poutcome	y	balance_level	\
0	unknown	may	1	0	unknown	0	2	
1	unknown	may	1	0	unknown	0	1	
2	unknown	may	1	0	unknown	0	1	
3	unknown	may	1	0	unknown	0	1	
4	unknown	may	1	0	unknown	0	1	
...	...	...	...	...	...	...	...	
45206	cellular	nov	3	0	unknown	1	1	
45207	cellular	nov	2	0	unknown	1	1	
45208	cellular	nov	5	3	success	1	2	

45209	telephone	nov	4	0	unknown	0	1
45210	cellular	nov	2	11	other	0	2

has_any_credit	
0	1
1	1
2	1
3	1
4	0
...	...
45206	0
45207	0
45208	0
45209	0
45210	0

[45211 rows x 15 columns]

```
[8]: # y sınıf oranlarını görelim
print(df["y"].value_counts(normalize=True))
```

```
0    0.883015
1    0.116985
Name: y, dtype: float64
```

Veri setimiz dengesiz bir yapıya sahip. Veri dengeleme yöntemlerinden RandomUnderSapmler yöntemi kullanılacaktır.

1.0.2 7.2. Modelin eğitilmesi

```
[9]: # Bağımlı ve bağımsız değişkenleri ayır
X = df.drop("y", axis=1)
y = df["y"]
```

```
[10]: # Kategorik verileri dönüştür
categorical_cols = X.select_dtypes(include="object").columns.tolist()
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
```

```
[11]: X_encoded
```

```
[11]:
```

	age	default	housing	loan	campaign	previous	balance_level	\
0	58	0	1	0	1	0	2	
1	44	0	1	0	1	0	1	
2	33	0	1	1	1	0	1	
3	47	0	1	0	1	0	1	
4	33	0	0	0	1	0	1	
...	...	...	...	...	...	...	...	

45206	51	0	0	0	3	0	1
45207	71	0	0	0	2	0	1
45208	72	0	0	0	5	3	2
45209	57	0	0	0	4	0	1
45210	37	0	0	0	2	11	2

	has_any_credit	job_blue-collar	job_entrepreneur	...	month_jul	\
0	1	0	0	...	0	
1	1	0	0	...	0	
2	1	0	1	...	0	
3	1	1	0	...	0	
4	0	0	0	...	0	
...	...	...	...	...	...	
45206	0	0	0	...	0	
45207	0	0	0	...	0	
45208	0	0	0	...	0	
45209	0	1	0	...	0	
45210	0	0	1	...	0	

	month_jun	month_mar	month_may	month_nov	month_oct	month_sep	\
0	0	0	1	0	0	0	
1	0	0	1	0	0	0	
2	0	0	1	0	0	0	
3	0	0	1	0	0	0	
4	0	0	1	0	0	0	
...	...	...	...	...	...	...	
45206	0	0	0	1	0	0	
45207	0	0	0	1	0	0	
45208	0	0	0	1	0	0	
45209	0	0	0	1	0	0	
45210	0	0	0	1	0	0	

	poutcome_other	poutcome_success	poutcome_unknown
0	0	0	1
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	1
...	...	...	...
45206	0	0	1
45207	0	0	1
45208	0	1	0
45209	0	0	1
45210	1	0	0

[45211 rows x 40 columns]

```
[12]: # Eğitim ve test verisi ayır
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.
↪3, random_state=42)
```

```
[13]: # Ölçekleme (özellikle mesafeye dayalı algoritmalar için faydalı)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[14]: # Dengesiz veriyi dengele
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train_scaled, y_train)
```

max_depth – Maksimum Derinlik : Ağacın derinliği sınırlandırılır. Çok derin ağaçlar veriye aşırı uyar ve overfitting yapar.

min_samples_split – Dallanma Sınırı: Bir dalın ikiye ayrılabilmesi için gerekli minimum örnek sayısı. Ne kadar yüksek olursa, model o kadar sade olur.

min_samples_leaf – Yaprak Düğümdeki Minimum Örnek Sayısı: Yapraklarda en az kaç örnek olmalı. Düşük değerlerde model ezberlemeye meyillidir.

max_leaf_nodes – Maksimum Yaprak Sayısı: Ağacın toplam yaprak düğüm sayısını sınırlandırır. Genellikle iyi bir sadeleştiricidir.

```
[15]: # Hiperparametre aralıkları
param_grid = {
    'max_depth': [3, 5, 8, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 5, 20],
    'max_leaf_nodes': [None, 5, 10, 20]
}
```

```
[16]: # Decision Tree ve GridSearchCV
dt = DecisionTreeClassifier(random_state=42)
grid_search = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1,
    verbose=1
)
```

GridSearchCV'nin karar ağacı modelini en iyi hiperparametrelerle eğitmesini sağlar.

- estimator=dt: Kullanılacak model (DecisionTreeClassifier) belirtir.
- param_grid=param_grid: Hiperparametrelerin aralığı (örneğin, max_depth, min_samples_split) belirtilir.

- `scoring='f1'`: Modelin başarısı F1 skoru ile ölçülür. (Precision ve Recall dengeyi)
- `cv=5`: 5 katlı çapraz doğrulama kullanılır, yani veri 5'e bölünür ve her bölümde model test edilir.
- `n_jobs=-1`: Tüm işlemci çekirdekleri kullanılarak paralel işlem yapılır.
- `verbose=1`: İşlem ilerlemesini gösterir, 1 daha az ayrıntı, 2 daha fazla ayrıntı verir.

Bu parametreler, GridSearchCV'nin karar ağacı modelini en iyi hiperparametrelerle eğitmesini sağlar.

```
[17]: # Grid Search modelini eđit
grid_search.fit(X_resampled, y_resampled)
```

Fitting 5 folds for each of 144 candidates, totalling 720 fits

```
[17]: GridSearchCV(cv=5, estimator=DecisionTreeClassifier(random_state=42), n_jobs=-1,
                  param_grid={'max_depth': [3, 5, 8, None],
                              'max_leaf_nodes': [None, 5, 10, 20],
                              'min_samples_leaf': [1, 5, 20],
                              'min_samples_split': [2, 5, 10]},
                  scoring='f1', verbose=1)
```

```
[18]: # En iyi model
best_model = grid_search.best_estimator_
print("\nEn iyi hiperparametreler:", grid_search.best_params_)
```

En iyi hiperparametreler: {'max_depth': 8, 'max_leaf_nodes': 20, 'min_samples_leaf': 20, 'min_samples_split': 2}

1.0.3 7.3. Model sonuçlarının değerlendirilmesi

```
[19]: # Test verisi üzerinde performans
y_test_pred = best_model.predict(X_test_scaled)

print("\nTEST SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_test, y_test_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_test_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_test_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_test_pred):.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

TEST SETİ PERFORMANSI:

Accuracy : 0.74

Precision: 0.25

Recall : 0.62
F1 Score : 0.36

Confusion Matrix:
[[9003 2963]
 [613 985]]

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.75	0.83	11966
1	0.25	0.62	0.36	1598
accuracy			0.74	13564
macro avg	0.59	0.68	0.59	13564
weighted avg	0.86	0.74	0.78	13564

```
[20]: # Eğitim seti performansı (Overfitting kontrolü)
y_train_pred = best_model.predict(X_resampled)
print("\nEĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):")
print(f"Accuracy : {accuracy_score(y_resampled, y_train_pred):.2f}")
print(f"Precision: {precision_score(y_resampled, y_train_pred):.2f}")
print(f"Recall : {recall_score(y_resampled, y_train_pred):.2f}")
print(f"F1 Score : {f1_score(y_resampled, y_train_pred):.2f}")
```

EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):
Accuracy : 0.70
Precision: 0.73
Recall : 0.64
F1 Score : 0.68

Bu sonuçlara göre, modelin overfitting yapmadığı, ancak hala iyileştirme yapabileceği bir durumda olduğunu söyleyebiliriz.

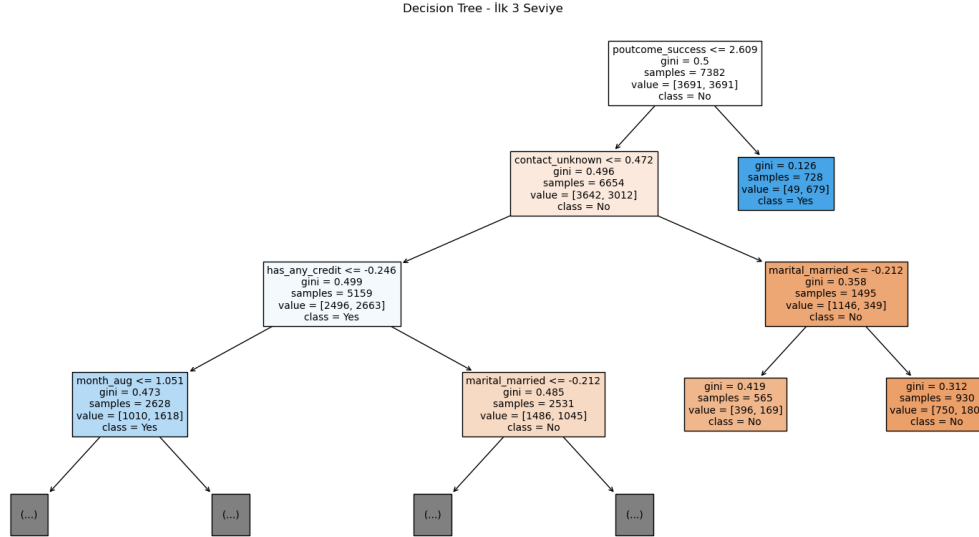
Eğitim Seti Performansı: - Accuracy (0.70): Eğitim setinde %70 doğruluk, modelin yeterince iyi performans gösterdiğini ancak bazı hatalar yaptığını gösteriyor. - Precision (0.73): Eğitim setinde doğru pozitiflerin oranı oldukça yüksek, bu modelin doğru sınıflandırma yapma eğiliminde olduğunu gösterir. - Recall (0.64): Model, gerçek pozitiflerin çoğunu yakalayabilmiş ancak yine de bazı doğru sınıflamaları kaçırmış. - F1 Score (0.68): Precision ve Recall arasında iyi bir denge olduğu söylenebilir, ancak F1 skoru daha da iyileştirilebilir.

Test Seti Performansı: - Accuracy (0.74): Test setinde doğruluk %74, eğitim setine oldukça yakın ve bu da modelin genelleme yeteneği açısından olumlu bir göstergedir. - Precision (0.25): Test setinde precision'ın düşük olması, modelin pozitif sınıfları doğru şekilde tanımlama konusunda sorun yaşadığını gösteriyor. Bu, çok fazla yanlış pozitif olduğunu işaret eder. - Recall (0.62): Recall'ın yüksek olması, modelin pozitif sınıfları tespit etme konusunda başarılı olduğunu, ancak yine de bazı pozitifleri kaçırdığını gösterir. - F1 Score (0.36): Test setindeki F1 skoru oldukça düşük. Bu, modelin hem yanlış pozitifleri (false positives) hem de yanlış negatifleri (false negatives) birlikte

yönetmede zorlandığını gösterir.

Özetle, modelin aşırı uyum yapmadığı, fakat precision ve F1 skoru gibi metriklerde daha fazla iyileştirme yapılması gerektiği söylenebilir.

```
[21]: # Decision Tree görselleştirmesi
plt.figure(figsize=(20, 10))
plot_tree(best_model,
          feature_names=X_encoded.columns,
          class_names=["No", "Yes"],
          filled=True,
          max_depth=3,
          fontsize=10)
plt.title("Decision Tree - İlk 3 Seviye")
plt.show()
```



```
[22]: # Cross Validation (5-Fold)
print("\nCROSS VALIDATION (5-Fold) Sonuçları:")
cv_scores = cross_validate(
    best_model,
    X_resampled,
    y_resampled,
    cv=5,
    scoring=['accuracy', 'precision', 'recall', 'f1'],
    return_train_score=True
)

# Ortalama skorları yazdır
```

```
for metric in ['train_accuracy', 'test_accuracy', 'train_precision',  
              'test_precision',  
              'train_recall', 'test_recall', 'train_f1', 'test_f1']:  
    mean_score = cv_scores[metric].mean()  
    print(f"{metric}: {mean_score:.2f}")
```

CROSS VALIDATION (5-Fold) Sonuçları:

train_accuracy: 0.71
test_accuracy: 0.70
train_precision: 0.75
test_precision: 0.74
train_recall: 0.62
test_recall: 0.60
train_f1: 0.68
test_f1: 0.66

Eğitim ve Test Seti Performansları: - train_accuracy (0.71) vs. test_accuracy (0.70): Eğitim ve test doğruluğu oldukça benzer. Bu, modelin overfitting yapmadığını ve eğitim verisine fazla uyum sağlamadığını gösterir. Model, hem eğitim hem de test verisinde benzer performans sergiliyor.

- train_precision (0.75) vs. test_precision (0.74): Precision (doğru pozitiflerin oranı) eğitim ve test setinde oldukça benzer. Model, pozitif sınıfları doğru şekilde tanımlama konusunda tutarlı bir şekilde performans gösteriyor.
- train_recall (0.62) vs. test_recall (0.60): Recall (gerçek pozitiflerin oranı) eğitim ve test setinde birbirine çok yakın. Model, gerçek pozitifleri yakalama konusunda benzer bir başarı sağlıyor. Ancak %60-62 arasındaki recall, modelin bazı pozitifleri kaçırdığını (false negatives) gösteriyor.
- train_f1 (0.68) vs. test_f1 (0.66): F1 skoru da eğitim ve test setinde birbirine yakın. Bu, modelin precision ve recall arasında iyi bir denge sağladığını ancak her iki metrikte de iyileştirme yapmaya ihtiyaç olduğunu gösterir.

Genel Olarak:

- Genelleme performansı oldukça iyi: Eğitim ve test seti sonuçları birbirine yakın, bu da modelin aşırı uyum yapmadığını ve iyi genelleme sağladığını gösterir.
- Precision ve Recall'un eğitim ve test setlerinde birbirine yakın olması, modelin her iki veri setinde de dengeli performans sergilediğini gösteriyor.
- F1 Skoru'nun biraz düşük olması (0.68-0.66 arası), modelin hem yanlış pozitifleri hem de yanlış negatifleri kontrol etmede yer yer zorlandığını gösteriyor.